

Association Rules on Online Retail: Evidence Before Cross-Sell

Student: Ivan Tsarikov | Team: Individual

Assignment: A04 - Association Rules (Week 4) | Date: 2026-10-06

Abstract - Retail basket associations can suggest cross-sells, but strong confidence can conceal limited actionability. I implemented complete Apriori and directed rule generation on 17,080 cleaned Online Retail invoices, using AI-assisted independent counting and browser checks. At 1% support and 60% confidence, 238 rules survive; cups to plates has 236 joint baskets and lift 43.83. I reject a 99.25% confidence herb-marker rule for narrow reach and saturation, showing why counts, baseline rates and experimental validation must guide action.

1. Introduction

In this homework, I investigate which products occur together in an invoice, quantify the strength and reach of each association, and decide whether it deserves a business experiment. A rule $A \rightarrow B$ describes B within baskets containing A ; it supplies neither purchase order nor proof that an intervention increases sales.

For example, stock 21086 is SET/6 RED SPOTTY PAPER CUPS and 21094 is SET/6 RED SPOTTY PAPER PLATES. Their matched design suggests a concrete cross-sell hypothesis. My AI-assisted solution implements the seven required functions while preserving the provided page, styling, dataset and helper code. I use independent verification results to assess useful and rejected patterns.

2. Related Work

Online Retail provides invoice-level product records from a UK retailer, with many wholesale customers [1]. Apriori separates frequent-itemset discovery from rule generation and uses downward closure to prune candidates [2]. I chose it because its levels and pruning steps are easy to check against exhaustive small-basket enumeration.

FP-Growth is a complete alternative using prefix compression and conditional trees [3]. The AI-assisted source review covered its method; I did not implement or benchmark it. A different miner must return the same frequent sets at the same threshold. I rejected pair-only enumeration because this homework includes larger sets. Co-purchase counts alone do not identify an offer effect; randomized offers can estimate that effect [4]. The source review also checked the starter paper attribution against publisher metadata [5].

3. Method



3.1 Population and basket construction

The UCI source has 541,909 line records [1]. The instructor export already removes cancellations, nonpositive quantities/prices, missing customer IDs/descriptions and non-product codes, then retains invoices with at least two distinct products. I use that supplied population unchanged: 17,080 baskets, 3,653 stock codes and 384,911 distinct invoice-product presences.

InvoiceNo defines a transaction, StockCode defines item identity and Description labels it. Repeated lines or multiple units contribute one presence. Separate invoices from the same customer stay separate. The logical $17,080 \times 3,653$ binary matrix has 62,393,240 cells and density 0.617%; basket sets and inverted postings represent it without allocating a dense matrix. The cleaned population is not all visits or all raw purchases.

3. Method (continued)

3.2 Exact counting and metric definitions

Let N be the number of input baskets; n_A , n_B and n_{AB} count baskets containing all of A , all of B and all of their union. Duplicates never add counts. Stock codes stay strings, and descriptions never determine identity. The counting function intersects postings starting with the rarest requested stock.

```
support(A -> B) = nAB / N
confidence(A -> B) = nAB / nA
lift(A -> B) = (nAB / nA) / (nB / N) = N x nAB / (nA x nB)
```

Metric helpers return {value, defined}. A zero denominator produces value 0 with defined=false, and the provided detail panel explains that the metric is undefined. Reversal keeps joint count, support and lift; confidence becomes n_{AB}/n_B . Empty count requests return 0 under the starter API; empty antecedents are excluded from rules.

3.3 Seven implemented functions

Function	Implementation and contract
dedupeBasket	Map strings/objects to stock codes; retain first-appearance order.
countItemset	Reuse raw-basket or supplied index; intersect deduplicated stock postings; empty/absent requests give 0.
computeSupport	Joint count / input N ; guard $N=0$.
computeConfidence	Joint count / antecedent count; guard $\text{count}(A)=0$.
computeLift	Divide defined confidence by consequent prevalence; guard undefined inputs and zero baseline.
findFrequentItemsets	Complete level-wise Apriori: canonical prefix joins, all-subset pruning, indexed parent intersections and safe early exit.
generateRules	All nonempty directed partitions, subset-count lookup, inclusive confidence filter; N inferred from exact count/support in the supplied itemsets.

3.4 Completeness, filters and configuration

A k -itemset yields $2^k - 2$ directed partitions before filtering. Recursion avoids a 32-bit bit-mask limit. The miner uses the input population, so the five-basket fixture does not inherit the retail N . No size cap, alternative metric or automatic lift filter is introduced in the generator.

The UI lists candidates meeting support/confidence thresholds. For downstream action analysis, I keep lift > 1 . I chose 1% support and 60% confidence: at least 171 joint invoices, with B in at least 60% of A invoices. This excludes the narrow 132-joint herb rule while retaining the 236-joint cups/plates hypothesis. At 3% support, the latter would be lost. These thresholds are a practical reach/screening choice, not proof of stability or profit.

Environment: classic browser JavaScript; Node.js 26.9.0 on Apple M4 Pro / arm64 for independent tests and timing; Apple Swift 6.3.3 with a fresh nonpersistent WKWebView for browser checks. Report generation uses Python 3.11 and ReportLab 5.0.1. No server, build step, notebook, Python generator or extra dependency is required to run the submitted app.

4. Experiments

4.1 Verification against independent expectations

Evidence	Observed result
Untouched provided starter	2 passed / 0 failed / 9 pending; Run rules reports the expected unimplemented-miner error.
Implemented provided harness	11 passed / 0 failed / 0 pending.
Independent Node suite	11/11 tests passed; 48 seeded random fixtures, 144 complete itemset comparisons and 432 rule comparisons.
Complete five-basket fixture	40% support: 9 sets, 12 rules; 20%: 19 sets, 38 rules, both at confidence $\geq 50\%$.
Higher-order coverage	All 30 directed partitions of a five-item set; real inspection sets reach size 6.
Real data and browser	Direct basket scans verify the three report examples. WKWebView loads file://, renders rules, selects/reverses cups/plates, updates sliders, shows empty/undefined states.

The oracle enumerates powersets and scans deduplicated baskets directly; it shares no Apriori joins, pruning or occurrence index with production. Checks cover stock identity, mixed object/string inputs, duplicates, unknown products, empty inputs, denominator guards, exact boundaries and input immutability. Browser tests use the real WebKit engine, not the Node DOM shim. I report these as agent-executed checks, separate from my personal arithmetic.

4.2 Threshold trade-off

Support	Confidence	Itemsets	Rules	Lift > 1	Time (s)
1.0%	30%	1,219	950	950	0.463
1.0%	60%	1,219	238	238	0.462
3.0%	30%	115	12	12	0.018
3.0%	60%	115	5	5	0.018
0.5%	10%	5,255	19,058	19,056	1.841

Table I. Single sequential Node measurements include mining, rule generation and enrichment; they exclude browser rendering and are not runtime guarantees. Full outputs are saved per setting.

At 1% support, raising confidence from 30% to 60% leaves 1,219 itemsets unchanged and reduces rules from 950 to 238. Raising support to 3% leaves 115 sets and only 5 rules at 60% confidence. Downward closure explains this: a larger set cannot be more frequent than a subset. Confidence acts after mining and removes directions, not itemsets.

All rules in the four 1%/3% runs have lift > 1. The largest consequent prevalence is $1,959/17,080 = 11.47\%$; thus 30% confidence implies lift at least 2.6156. A near-independent rule cannot pass that confidence filter. The separate 0.5%/10% inspection run finds 19,058 candidates, including two lift-below-1 patterns; only 19,056 are positive. This inspection pool is not the chosen final table.

The recorded WebKit default 1%/30% table has 950 rows, and my chosen 1%/60% table has 238. Displayed cups/plates values agree with the independent counts and my rounded calculations. I preserved the dataset, page and stylesheet; the README now documents my repository and web-app link.

5. Business and Algorithmic Analysis

5.1 Useful rule and direction

A: 21086 - SET/6 RED SPOTTY PAPER CUPS.

B: 21094 - SET/6 RED SPOTTY PAPER PLATES.

Rule	N	nA	nB	nAB	Support	Confidence	Lift
Cups -> plates	17,080	283	325	236	1.38%	83.39%	43.8258
Plates -> cups	17,080	325	283	236	1.38%	72.62%	43.8258

Plates occur in only 1.90% of retained invoices overall, compared with 83.39% of cups invoices. The joint count of 236 exceeds the 171-invoice cutoff. I would investigate a matching-plate recommendation when cups are in a live cart and plates are absent. The counts support this hypothesis, but do not show incremental sales. The different marginal counts explain why the reverse confidence is lower.

5.2 High confidence can still be unhelpful

A: HERB MARKER THYME (22916), ROSEMARY (22917), MINT (22919), BASIL (22920) and CHIVES (22921). B: HERB MARKER PARSLEY (22918). Across N=17,080, nA=133, nB=187 and nAB=132: support 0.7728%, confidence 99.25%, lift 90.6502; B baseline 1.0948%.

The naive reading is an almost-certain sixth-marker cross-sell. The association is real and positive, but the five-product trigger reaches only 133 completed invoices, and 132 already contain B. Only one completed antecedent invoice lacks it. The rule fails my 1% support requirement and has little observed unsatisfied reach, so I reject it for immediate broad placement. Those completed-invoice counts are not live-cart exposure counts or a causal opportunity estimate.

A separate baseline trap is 22726 (GREEN BAKELIKE ALARM CLOCK) -> 85123A (WHITE HANGING HEART T-LIGHT HOLDER): nA=784, nB=1,959, nAB=89, support 0.5211%, confidence 11.35%, lift 0.9898. A buyer might mistake 89 co-purchases for useful targeting, but confidence is below B's 11.47% baseline. I reject it for no positive association; a lift this close to 1 does not establish strong inhibition.

5.3 Use case, limitation and causality

I propose testing one matching-plate recommendation in eligible cups carts, suppressing it when plates are already present. The export excludes abandoned baskets, non-purchasing visits, guest orders and single-product invoices; its wholesale/retail mixture can also differ from a deployment audience. Completed baskets lack purchase sequence and recommendation exposure.

My explanation is that customers planning a party may need both matching cups and plates. Shared occasion can drive both purchases; promoting cups is not shown to cause plate sales. Lift describes observed conditional versus baseline frequencies. A randomized recommendation offer tests an intervention, not the causal effect of merely purchasing A [4].

5. Analysis and Discussion (continued)

5.4 Validation before action

I would freeze the cups/plates rule and obtain a later dated invoice period under the same cleaning definition. I would recompute N, counts, baseline prevalence, support, confidence and lift, and examine seasonal and customer-mix changes. My screening decision is to retain the rule only if later support $\geq 1\%$, confidence $\geq 60\%$ and lift > 1 , with uncertainty reported. Later customer IDs would allow dependence-aware uncertainty checks. The export lacks usable per-invoice timestamps, so I have not performed this stability test.

To estimate incremental effect, I would randomly assign eligible live shoppers with cups and no plates to a matching-plate offer or control. I would define attach rate as the share of all eligible assigned shoppers completing a plate purchase, including non-purchasers in the denominator. Sample size would depend on a live pilot and a commercially meaningful minimum effect. I would accept the rule only if a prespecified effect/uncertainty criterion and conversion, revenue or margin guardrails are met. This needs new cart, assignment, outcome and commercial data; I have not run this A/B test [4].

5.5 Failure counterexample, root cause and correction

A naive integer shortcut fails an exact support boundary. In JavaScript, 0.07×100 evaluates to 7.000000000000001 , so $\text{ceil}(\text{support} \times N)$ would require 8 baskets and wrongly reject a set present in exactly 7 of 100. Floating-point multiplication is the cause. My implementation checks whether the preceding integer count already meets $\text{count}/N \geq \text{support}$ before using the cutoff for early termination. The agent-executed regression confirms that A, B and AB survive 0.07 and disappear at 0.07000000000000002 , matching an exhaustive oracle.

The AI-assisted source review also found the original README's wrong journal and "Sai Liang Sain" attribution. Publisher metadata gives Sai Laing Sain and Journal of Database Marketing & Customer Strategy Management [5]. My report and revised README use the corrected attribution; the supplied dataset is unchanged. The initial 2/0/9 result is expected unfinished scaffolding, not an invented hidden implementation failure.

What worked for my solution: exact contracts, indexed counts and an oracle independent of the chosen miner. My main takeaway is that every default candidate already has positive lift, while a 99% rule can have almost no unsatisfied completed-basket reach. My next improvement would be to collect dated cart/exposure data and run the preregistered plate-offer experiment to address these actionability limits.

6. AI Usage Disclosure

Tools and scope. I used OpenAI Codex VS Code client 0.159.2 and delegated Codex agents for source research, the seven-function implementation, independent tests, experiments, browser automation, analysis and PDF preparation. I do not infer an exact backend service build. Agent tests, direct scans and source readings are recorded in the native session and working evidence files.

What I personally verified. From the independently checked counts supplied by the agent, I calculated: $236/17,080 \times 100 = 1.38\%$; $236/283 \times 100 = 83.39\%$; $236/325 \times 100 = 72.62\%$; lift = $(236 \times 17,080)/(283 \times 325) = 4,030,880/91,975 = 43.83$. I explained shared party intent, why association does not establish a promotion effect, and the exclusion of abandoned/non-purchasing visits. The agent compared my rounded results with the independently counted data and the exact page values.

Limits and audit trail. The agent supplied and checked the raw counts. My personal verification was the arithmetic and explanation; the agent ran the tests, scanned the baskets, operated the browser and read the relevant sources. The RFM paper was checked only for metadata/abstract context. Future stability and causal results remain unverified. I include `ivan_tsarikov_a04_session.json` as an unmodified native JSONL snapshot; the supplemental session-logs archive preserves delegated tool records. My input and machine checks remain distinguishable.

References

- [1] D. Chen, Online Retail [Dataset], UCI Machine Learning Repository, 2015, doi: 10.24432/C5BW33. [Online]. Available: <https://archive.ics.uci.edu/dataset/352/online%2Bretail>. [Accessed: Oct. 6, 2026].
- [2] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), Santiago, Chile, 1994, pp. 487-499. [Online]. Available: <https://www.vldb.org/conf/1994/P487.PDF>. [Accessed: Oct. 6, 2026].
- [3] J. Han, J. Pei and Y. Yin, "Mining frequent patterns without candidate generation," in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 1-12, doi: 10.1145/342009.335372. [Online]. Available: <https://www.cs.sfu.ca/~jpei/publications/sigmod00.pdf>. [Accessed: Oct. 6, 2026].
- [4] M. A. Hernán and J. M. Robins, Causal Inference: What If. Boca Raton, FL, USA: Chapman & Hall/CRC, 2020. [Online]. Available: <https://miguelhernan.org/whatifbook>. [Accessed: Oct. 6, 2026]. The authors' linked 2026-08-19 reading version retains the prescribed 2020 citation.
- [5] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," Journal of Database Marketing & Customer Strategy Management, vol. 19, no. 3, pp. 197-208, 2012, doi: 10.1057/dbm.2012.17. [Online]. Available: <https://link.springer.com/article/10.1057/dbm.2012.17>. [Accessed: Oct. 6, 2026].

For this report, I used the agent's reading of relevant substantive sections of entries [1]-[4]. Entry [5] supports the metadata correction; its full text was not read. I retain complete experiment outputs and source-access notes as local evidence, and provide the app, report and native session records as the submission artifacts.